

README — text preparation, model tokenization, and LSA

This takes a sample produced by the corpus-loading-and-sampling step and gets it ready for a language model: it cleans the text minimally, then either tokenizes it with a contextual model (BERT, GPT-2, ...) aligning each target word with its tokens, or estimates a static LSA representation.

This folder contains:

File	What it is for
<code>text_prep.py</code>	Minimal, documented text cleaning, keeping offsets correct.
<code>tokenization.py</code>	Loads a contextual model's tokenizer, tokenizes, aligns the segment with its tokens.
<code>lsa.py</code>	Static representation: term-document matrix + SVD (one vector per word).
<code>tracelog.py</code>	The audit trail (a copy lives here so the folder is self-contained).
<code>exe.py</code>	The interactive program that runs everything here.

Folder layout

This folder and the sampling folder sit side by side inside one parent folder:

```
parent/  
├─ corpus_loading_and_sampling/  (exe.py, ..., samples/)  
└─ text_prep_model_tokenization/ (this exe.py, text_prep.py, ...)
```

This program finds the samples automatically in the sibling folder. Run it with `python exe.py` from a terminal. You need `pip install transformers scikit-learn` (transformers for the contextual models, scikit-learn for LSA).

1. `text_prep.py` — minimal text preparation

Motivation. Keep the text as produced; change it only with an explicit, recorded reason. The delicate part is that whenever the text changes, a word's **offset** can shift — so this module recomputes offsets to keep them pointing at the right place.

Transformations, by how safe they are:

- **Safe (on by default):** `unicode_nfc` (unify equivalent Unicode forms) and `collapse_whitespace` (multiple spaces/newlines → one). These remove technical noise without losing meaning.

- **Risky (off by default):** `lowercase` and `strip_accents`. These can erase informative variation (proper-noun casing, "papá" vs "papa"), so they must be enabled explicitly and are flagged in the tracelog.

Functions

- `prepare_text(text, steps, offset)` — Applies the named transformations to one text, carrying an offset through so it stays correct.
 - `prepare_units(units, steps, save_to, ...)` — Prepares every unit of a sample. `units` can be a file path (the sample JSON, read automatically) or a list. Adds `prepared_text` and the list of `transformations`; cleans the `left`/`right` context too. Saves the output and records the decision.
 - `load_prepared(path)` — Reads prepared units back.
 - `SAFE_DEFAULTS` — the two safe steps used when you don't specify any.
-

2. `tokenization.py` — contextual model: tokenize + align

Motivation. A model doesn't split text into words but into sub-word **tokens** ("yellow" might become "yel"+"low"). To later read the vector of *your* word, you must know which tokens are the target. The offsets carried from the sampling step are exactly what makes that alignment possible.

The model is a **parameter** (default `bert-base-uncased`); change it to `gpt2`, BETO, XLM-RoBERTa, or any Hugging Face id without touching the code. Only the tokenizer is loaded here, not the heavy weights.

Functions

- `load_tokenizer(model_name)` — Loads a model's tokenizer (downloads it the first time).
 - `align_offset_to_tokens(offsets_mapping, segment_offset)` — The key piece: returns which token positions overlap the segment. Handles words split into several sub-tokens, and skips special tokens.
 - `tokenize_units(units, model_name, ...)` — Tokenizes each unit and, for **segments**, aligns the word with its tokens (`segment_token_indices`). **Detects the unit type automatically:** sentences and documents are tokenized whole, with no segment alignment (the whole unit is the observation). `units` can be a file path or a list. Saves and logs.
 - `DEFAULT_MODEL` — the default model id.
-

3. `lsa.py` — static representation (LSA)

Motivation. The static, distributional alternative to BERT/GPT. Unlike contextual models (a word's vector depends on its sentence), LSA gives **one vector per word**, the same everywhere, built from how words co-occur across documents.

How it works: build a term-document matrix (TF-IDF by default) → reduce it with Truncated SVD to a few latent dimensions → each word becomes a dense vector.

Functions

- `estimate_lsa(documents, target_words, n_components, ...)` — Builds the matrix and reduces it. Returns the vocabulary, the word vectors, the target words' vectors, and how much variance the reduction kept. Needs at least 2 documents. Saves and logs.
 - `documents_from_prepared(prepared)` — Recovers one text per document from the prepared units (LSA works at the document level).
-

4. `tracelog.py`

A copy lives here so this folder is self-contained. Records every decision made here (text preparation, tokenization or LSA) with its reason.

5. The exe (`exe.py`) — the interactive program

Motivation. Ties everything here together: finds a sample, prepares the text (and saves it), then builds a representation (and saves that), recording everything.

What it produces

Outputs go to `output/<run>/` inside this folder:

File	Contents
<code>prepared.json</code>	The text BEFORE the model (after minimal cleaning).
<code>model_tokens.json</code>	If you chose a contextual model: tokens + alignment.
<code>lsa_space.json</code>	If you chose LSA: the word vectors and the space.
<code>tracelog_part2.json</code>	The record of every decision made here.

Every question, in order

1. **Samples folder** — it auto-finds the `samples` folder in the sibling folder, shows it, and lets you **press Enter to accept or type another path**.
2. **Which sample to process** — a menu of the available runs (one sample per run folder). It lists the final sample of each run (the kwic file, or the sentence/document sample), not the intermediate sub-corpus.
3. **Text preparation level** — *safe only* (recommended), or also enable the risky steps (lowercase / strip accents), which are flagged in the tracelog.
4. **Which representation:**
 - **contextual model** — then a **menu of models** (you don't type the id): BERT (uncased/cased), RoBERTa, DistilBERT, GPT-2, BETO (Spanish), multilingual BERT, XLM-RoBERTa, an MPNet sentence model, or "other" to type an id. It tokenizes and aligns, saving `model_tokens.json`.
 - **LSA** — then the **number of dimensions** (default 100). It estimates the space, saving `lsa_space.json`.

At the end it prints the **tracelog** and lists the files written.

Two honest notes

- **MPNet / sentence-transformers** give one vector *per sentence*, not per token aligned to your word. They fit the **sentence** unit better than the **segment** unit. For analyzing a word in context, BERT / RoBERTa / BETO / XLM-R are the natural choices.
- **First use downloads the model** (BETO and XLM-R are a few hundred MB), so the first run with a given model is slow; later runs use the cache.

How it chains together

```
sample           → samples/<term>_<unit>/stage2_kwic.json
                  |
text_prep         → output/<run>/prepared.json           (text before the model)
                  |
├ contextual      → output/<run>/model_tokens.json       (tokens + alignment)
└ LSA             → output/<run>/lsa_space.json           (word vectors)
                  |
                  → tracelog_part2.json   (every decision, with its reason)
```

Each step reads the previous file, saves its own, and logs to the tracelog — so everything, from the sample to model-ready data, stays traceable.